

Дополнительная лекция по Си № 6

30 апреля 2025 г.

Коновалов А.В.

Раздельная компиляция

Язык Си допускает раздельную компиляцию — написание программы, состоящей из нескольких файлов исходного текста, которые компилируются независимо.

Файлы исходного текста с расширением `.c` сначала компилируются в объектные файлы — имеют расширение `.o` (на платформе Unix и у портированных с неё компиляторов, например, GCC) или `.obj` (на платформе Windows).

Причём, на платформе Unix есть ещё и промежуточный этап — генерация ассемблера: исходник сначала компилируется в код на ассемблере (расширение `.s`), затем, ассемблерный код преобразуется в объектный файл. Но нас сегодня это не интересует.

Затем, после получения объектных файлов выполняется этап компоновки (linking, «линковка») — содержимое объектных файлов + файлов библиотек объединяется, внешние ссылки разрешаются, получается исполнимый модуль операционной системы (`a.out`, `a.exe`).

Почему объектный файл не является окончательным и нужен этап компоновки. В объектном файле могут присутствовать т.н. внешние ссылки — ссылки на скомпилированные сущности (функции, глобальные переменные...), которые в данном файле отсутствуют, но должны присутствовать в других объектных файлах. На стадии компоновки эти ссылки разрешаются.

Скажем, если в нашей программе вызывается функция `printf`, то в коде, содержащемся в объектном файле, будет в инструкции вызова функции вместо адреса `printf` нули, но в отдельной секции файла будет ссылка на место, указывающая, что сюда нужно будет вписать актуальный адрес функции `printf` на этапе компоновки. Ещё, кстати, компоновщик иногда называется «link editor», т.е. редактор связей.

Библиотека (имеет расширение `.a` на Unix или `.lib` на Windows) это просто несжатый архив объектных файлов.

Раздельная компиляция в языке Си

Глобальные сущности (функции и глобальные переменные) в программе на Си могут иметь внешнюю и статическую компоновку. Сущности с внешней компоновкой доступны для обращения из других единиц трансляции (т.е. функцию можно описать в одном файле, а вызвать в другом). Сущности со статической компоновкой из других единиц трансляции недоступны.

Единица трансляции (translation unit) — то, из чего собирается объектный файл. Для языка Си это исходный текст после выполнения препроцессирования.

Внешняя компоновка в языке Си используется по умолчанию — когда мы описываем глобальную переменную или функцию, они уже имеют внешнюю компоновку.

Статическая компоновка осуществляется с использованием ключевого слова `static`:

```
int x; /* внешняя компоновка */
static int y; /* статическая компоновка */

/* внешняя компоновка */
int foo(int a) {
    return a + 1;
}

/* статическая компоновка */
static int bar(int b) {
    return b * 2;
}
```

А как же воспользоваться сущностью со внешней компоновкой из другой единицы трансляции? Её нужно объявить.

В языке Си различают *объявление* и *определение*. **Объявление (declaration)** сообщает компилятору о том, что некоторое имя существует, но описывает как оно реализовано. **Определение (definition)** уже описывает реализацию.

```
/* объявления */
int f(int x, int y);
struct S;

/* определения */
int f(int x, int y) {
    return x + y;
}

struct S {
    double a, b;
}
```

```
};
```

Объявление функции даёт компилятору достаточно сведений, чтобы скомпилировать вызов функции: сколько параметров, какого типа, каков тип возвращаемого значения.

Объявление структуры говорит о том, что такой тег структуры есть — можно, например, описать переменную типа «указатель на структуру» или объявить функцию с параметрами типа структура.

Объявление структуры в языке Си может быть совмещено с другой конструкцией:

```
struct A foo(int a, double b);  
struct B *p;
```

Здесь объявлена функция `foo` и определён (или объявлен?) указатель `p` и одновременно с этим объявлены две структуры: `A` и `B`. А указатель `p` определён или объявлен? Об этом чуть позже.

Для каждой именованной сущности в единице трансляции должно быть не более одного определения, однако объявлений может быть сколько угодно.

В языке Си (в отличие, замечу, от C++) действует такое правило: описание переменной без инициализатора может быть и объявлением, и определением. Описание переменной с инициализатором — всегда определение. Если переменная объявлена один или несколько раз, но без инициализатора, какое-то одно из объявлений будет считаться определением, остальные объявлениями:

```
int x;          // объявление  
int x = 10;     // определение  
int x;          // объявление  
  
/* какое-то из этих двух объявлений — определение */  
int y;  
int y;
```

Заметим, что в C++ код выше приведёт к ошибке — компилятор C++ увидит три *определения* переменной `x` и два — `y`. В Си же многократное объявление переменной — не ошибка.

Пример. Опишем кольцевой список из двух глобальных переменных:

```
struct List {  
    int value;  
    struct List *next;  
};  
  
struct List nodeA; /* объявление */  
  
struct List nodeB = { /* определение */  
    .value = 100,  
};
```

```

        .next = &nodeA,
    };

    struct List nodeA = { /* тоже определение */
        .value = 500,
        .next = &nodeB,
    };

```

Это будет ошибкой:

```

int x = 100;
int x = 500;

```

Чтобы сослаться на функцию, определённую в другой единице трансляции, нужно её просто объявить:

```

int foo(double x, int *y);

```

А чтобы описать переменную, определённую в текущей единице трансляции, так просто объявить её не получится:

```

int x;

```

т.к. это уже будет определением переменной, компоновщик выдаст ошибку.

Компоновщик выдаёт ошибку, если видит определение (вернее, для него это внешнее имя) в нескольких единицах трансляции.

Для объявления внешних переменных (и функций тоже) используется ключевое слово `extern`:

```

extern int mode;
extern void g(int *src, int *dest);

```

Запись ключевого слова `extern` необходима для объявления внешних переменных, для функций её писать не обязательно, но иногда пишут из соображений стиля — подчеркнуть, что это не предобъявление функции, определённой ниже, а именно ссылка на внешнюю функцию.

Ключевое слово `extern` используется только для функций и переменных, т.к. именно для них порождается код (для функций код процессора, для переменных — начальное значение). Для описаний типов ничего не порождается, к ним `extern` не применимо.

У статических переменных и статических функций тоже могут быть объявления, просто в начало добавляется `static`:

```

/* объявления */
static int x;
static void f();

/* определения */
static int x = 10;

```

```
static void f() {
    printf("Hello!");
}
```

Если в единице трансляции статическая функция объявлена, но не определена, компиляция завершится с ошибкой.

Согласованность объявлений и определений

Объявление внешней сущности в одной единице трансляции и определение в другой должны совпадать — эта должна быть одна и та же сущность (функция или переменная), у них должен быть одинаковый тип, для функций — одинаковая сигнатура.

Иначе, когда компилятор будет генерировать машинный код для исходного кода, обращающегося к внешней сущности (например, вызывая функцию), он будет обращаться к ней не так, как надо.

Рассмотрим пример:

```
/* Файл a.c */
char message[] = "Hello!";

/* Файл b.c */
#include <stdio.h>

extern char *message;

int main() {
    printf("%s\n", message);
    return 0;
}
```

В файле `a.c` описана область памяти со внешним именем `message` размером 7 байтов (6 байт на знаки `Hello!` + 1 байт на концевой ноль). Содержимое переменной `message` — `Hello!\0`.

Файл `b.c` ожидает, что есть внешнее имя `message` (в какой-то другой единице трансляции), но предполагает, что `message` имеет размер 4 или 8 байт (в зависимости от разрядности целевой платформы) и хранит адрес некоторой строки. На 32-разрядной платформе как адрес проинтерпретируются байты `Hell`, что может соответствовать невыделенной области памяти (получим `SEGFAULT`). На 64-разрядной платформе при чтении содержимого переменной `message` мы вообще выйдем за пределы допустимой области (прочитаем какой-то соседний байт) и в любом случае получим некорректный адрес.

Т.е. в обоих случаях знаки внутри строки будут ошибочно проинтерпретированы как байты адреса.

В правильной программе нужно сделать или так:

```
/* Файл a.c */
char message[] = "Hello!";
```

Тогда переменная `message` будет хранить адрес начала массива `char`'ов.

Или так:

```
/* Файл b.c */
#include <stdio.h>

extern char message[];

int main() {
    printf("%s\n", message);
    return 0;
}
```

Тогда в файле `b.c` мы будем ожидать, что `message` — это массив, а не адрес.

Обеспечивать согласованность объявлений и определений в единицах трансляции вручную мучительно: при изменении определения нужно вносить правки в кучу других файлов. Для автоматического согласования уже давно придуман инструмент — препроцессор для языка Си.

Использование заголовочных файлов

Можно описать необходимые объявления в одном файле и включать содержимое этого файла в те единицы трансляции, где эти объявления требуются.

Включение объявления осуществляется препроцессором при помощи директивы `#include`:

```
#include <stdio.h>
#include "matrix.h"
```

Включаемый файл называется **заголовочным** и как правило имеет расширение `.h`.

Угловые кавычки говорят о том, что заголовочный файл (например, `stdio.h`) следует искать в системных каталогах, прямые кавычки — в текущей папке.

Заголовочные файлы зачастую включают не только объявления переменных и функций со внешней компоновкой, но и определения структур, объединений, перечислений, макросов и т.д.

На повторные определения компилятор выдаёт ошибку, поэтому при повторном включении заголовочного файла можно получить ошибки компиляции. Повторное включение может быть не непосредственным, а косвенным. Например, файлы `a.h` и `b.h` требуют определения структуры, присутствующей

в файле `c.h`, их одновременное включение приведёт к повторному включению файла `c.h`.

Переносимым способом борьбы с повторным включением служит приём (идиома), который называется **стражи включения** или **защита включения** (include guards):

```
#ifndef FILENAME_H
#define FILENAME_H

<содержимое заголовочного файла>

#endif /* ifndef FILENAME_H */
```

Заголовочный файл окружается директивами `#ifndef FILENAME_H` и `#endif` (т.е. первая и последняя строки), что при определённом макросе `FILENAME_H` предотвращает добавление в единицу трансляции содержимого файла. Внутри этих директив макрос `FILENAME_H` определяется. Таким образом, при первом включении, когда макрос не определён, содержимое файла включается и макрос определяется. При повторных включениях содержимое файла уже не включается.

Макросам-стражам включения обычно дают имена, совпадающие с именем включаемого файла и записанными большими буквами. Есть плохая практика добавлять в начало прочерк, но эта практика плохая, т.к. согласно Стандарту языка Си (и C++ тоже) имена, начинающиеся на прочерк, зарезервированы.

Заголовочный файл с объявлениями включается не только в те единицы трансляции, где эти объявления используются, но и в исходный файл, где эти сущности уже определяются. На первый взгляд избыточно, но это позволяет контролировать ошибки: если объявление и определения не совпадают, то произойдёт ошибка компиляции, что гарантирует согласованность объявлений и определений.